CHƯƠNG 3:
LỚP VÀ ĐỐI TƯỢNG

LỚP VÀ ĐỐI TƯỢNG

1. ***Các khái niệm:*** Instance; property, đối tượng, biến class, biến instance, biến local, new, this, lớp trừu tượng.
2. ***Phương thức:*** Các loại tham số trong phương thức, kết quả trả về của phương thức.

LỚP VÀ ĐỐI TƯỢNG

❖ Đối tượng (Object)

- Là các **sự vật** trong thế giới thực: Bàn, ghế, xe,…
- Là các **khái niệm** như: Khoa, phòng ban,…
- Có các thuộc tính và hành vi.



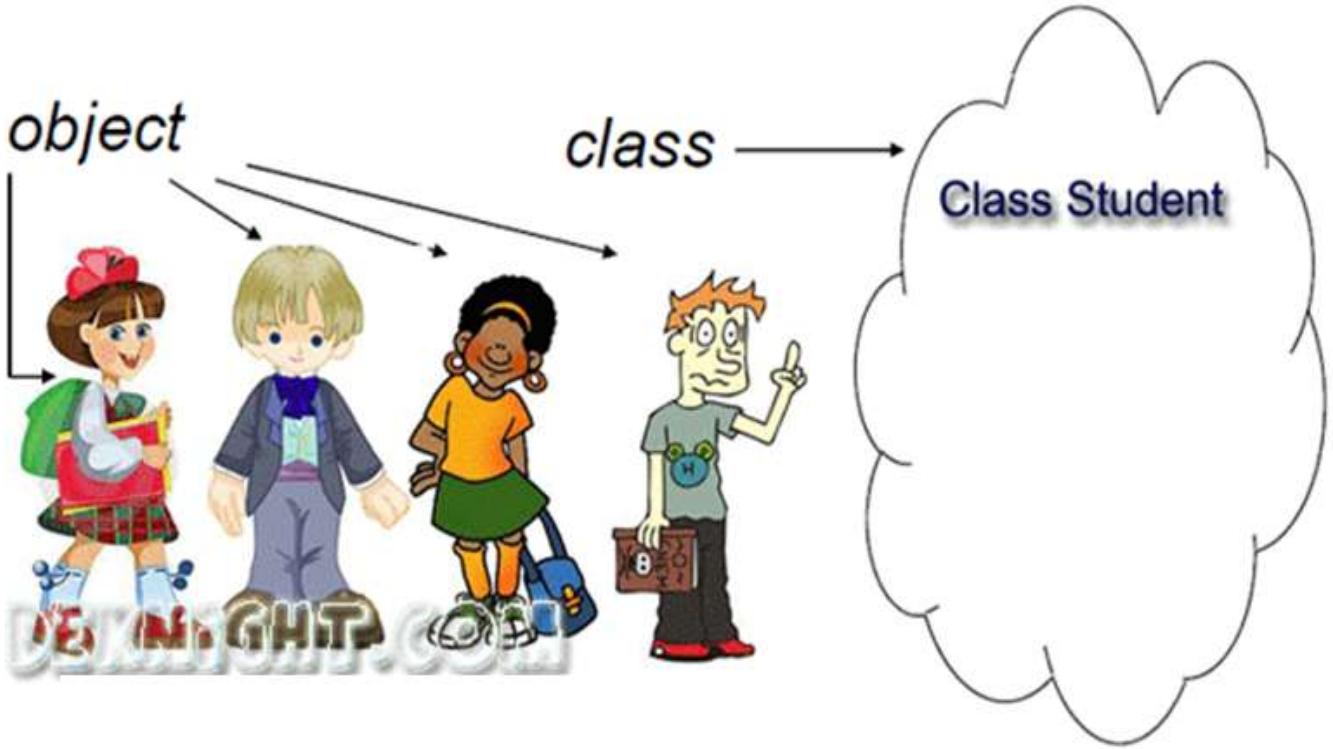
LỚP VÀ ĐỐI TƯỢNG

❖ Lớp (Class)

- Các đối tượng có đặc tính tương tự nhau được gom chung lại thành lớp đối tượng. Ví dụ **NGƯỜI** là một lớp đối tượng.
- Một lớp đối tượng được đặc trưng bởi các **thuộc tính** và các **hoạt động** (hành vi, thao tác)

LỚP VÀ ĐỐI TƯỢNG

❖ **Phân biệt giữa đối tượng và lớp**



LỚP VÀ ĐỐI TƯỢNG

❖ Các thành phần của lớp

- **Tên lớp:** duy nhất, dùng phân biệt lớp này với lớp khác trong cùng một phạm vi.
- **Thành phần dữ liệu, thuộc tính:** là các thành phần cấu tạo nên đối tượng, đặc trưng cho đối tượng.
- **Phương thức:** các hoạt động, của đối tượng thuộc lớp.

➢ ***Instance:*** Một đối tượng được tạo ra từ một lớp được gọi là một thể hiện (instance) của lớp đó.

LỚP VÀ ĐỐI TƯỢNG

❖ Các thành phần của lớp

Ví dụ: Lớp PhanSo gồm các thành phần:

- **Thuộc tính:**
  - Tử số
  - Mẫu số
- **Phương thức:**
  - Nhap(): nhập dữ liệu cho phân số
  - Xuat (): xuất dữ liệu cho phân số
  - ….

VÍ DỤ

Biến thuộc tính

Phương thức

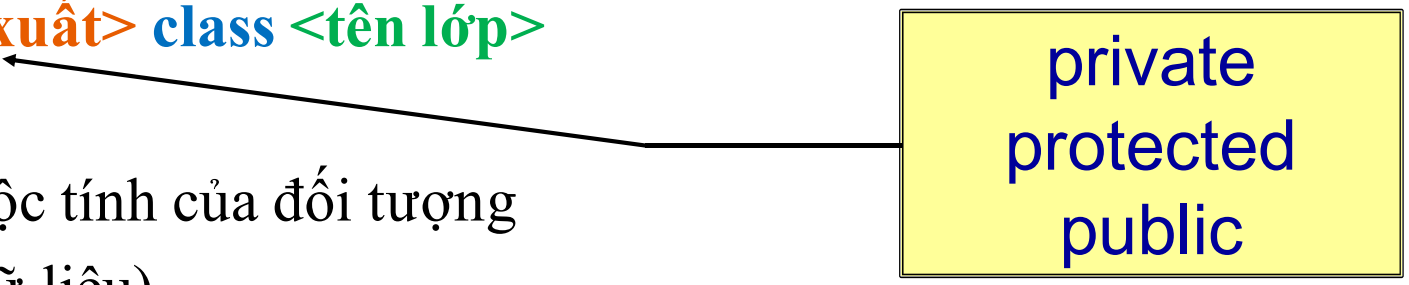
```
class HinhChuNhat
{
    private float dai;
    private float rong;
    public void nhap()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Nhap dai");
        dai= sc.nextFloat();
        System.out.println("Nhap rong");
        rong= sc.nextFloat();
    }
    public float dientich()
    {
        return dai*rong;
    }
    public void xuat()
    {
        System.out.println("Dien tich HCN: "+ dientich());
    }
}
```

LỚP VÀ ĐỐI TƯỢNG

- Cú pháp khai báo lớp:

```
<thuộc tính truy xuất> class <tên lớp>
{
        //khai báo thuộc tính của đối tượng
        (thành phần dữ liệu)

        //khai báo phương thức
        (thành phần hàm)
}
```



LỚP VÀ ĐỐI TƯỢNG

THUỘC TÍNH TRUY XUẤT (ACCESS MODIFIER)

- **Private:** chỉ được truy xuất bên trong lớp đó
- **Public:** ở đâu cũng truy xuất được
- **Protected:** chỉ được truy xuất ở trong cùng package và lớp con
- **Default:** được truy xuất trong cùng package

BIẾN CỤC BỘ

- Được khai báo bên trong các method, constructor, hoặc block.
- Biến Local chỉ xuất hiện trong giới hạn của method, constructor hoặc block.
- Biến Local sẽ bị hủy ngay khi thực hiện xong method, constructor hoặc block.
- Trước khi sử dụng, các biến cục bộ phải được gán giá trị mặc định.
- Biến cục bộ được tạo ra sẽ lưu trong vùng nhớ stack của bộ nhớ

BIẾN INSTANCE

- Được khai báo trong class, nhưng bên ngoài các method, constructor hoặc block.
- Được tạo khi đối tượng được tạo, có thể tạo ra đối tượng bằng cách sử dụng từ khóa new.
- Khi đối tượng bị hủy thì biến instance cũng sẽ bị hủy.
- Biến instance có thể được gọi trực tiếp trong lớp mà nó được khai báo bằng tên.
- Có giá trị mặc định: Number = 0, boolean = false, object = null.

Biến static

- Được khai báo với từ khóa static trong class nhưng bên ngoài method, constructor hoặc block
- Biến static là duy nhất bên trong một class.
- Biến static hiếm khi được sử dụng, chủ yếu nó được dùng để khai báo các constants (hằng).
- Được tạo ra khi chương trình bắt đầu chạy và hủy khi chương trình kết thúc.
- Giá trị mặc định của biến static phụ thuộc vào kiểu dữ liệu khai báo tương tự biến instance.
- Có thể được truy xuất bằng cách gọi tên lớp: ClassName.VariableName

Ví dụ

```
public class Bien {
public static float bi = 3.14;// Biến static
int n; // Đây là biến instance
public Bien () {
   char c = 'c'; // Đây là biến local
   }
}
```

Ví dụ

```
public class Sinhvien {
public static String ten = "Nguyen Van A"; // biến static ten
public static int tuoi = 21; // biến static tuoi
public static void main(String args[]) {
   // Gọi biến static trực tiếp
   System.out.println("Ten : " + ten);
   // Gọi biến static bằng cách gọi thông qua tên class
   System.out.println("Tuoi : " + Sinhvien.tuoi);
   }
}
```

Khởi tạo một đối tượng

➢ ***New:*** Dùng để khởi tạo một đối tượng

Đoạn mã sau đây sử dụng lớp employee để tạo một nhân viên sau đó gọi các phương thức của lớp.

```
public static void main(String[] args) {

    Employee emp = new Employee();
    emp.input();
    emp.output();
}
```

- Toán tử New sử dụng để tạo một đối tượng
- Biến emp chứa tham chiếu tới đối tượng
- Sử dụng dấu chấm (.) để truy xuất đến các thành phần của lớp (thuộc tính và phương thức)

Từ khóa This

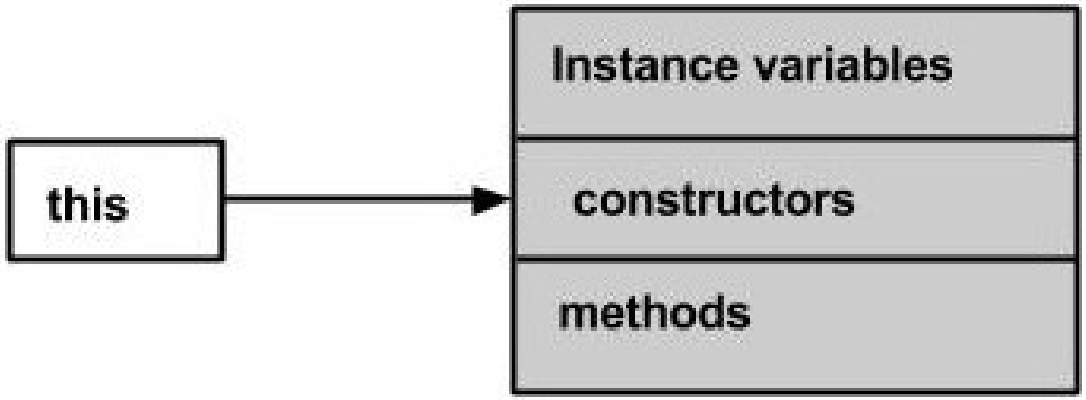
- **This** là một tham số ngầm định trong các phương thức của lớp (trừ phương thức static).
- Nếu biến được định nghĩa trong thân hàm là biến cục bộ chỉ tồn tại khi hàm được gọi. Khi biến này đặt tên trùng với biến đối tượng hoặc biến lớp, nó sẽ che khuất biến đối tượng hay biến lớp trong thân hàm.
- Từ khoá this có thể dùng bên trong bất cứ phương thức nào để tham chiếu đến đối tượng hiện hành, khi biến đối tượng trùng tên với biến cục bộ.

Từ khóa this

this là từ khóa được sử dụng để tham chiếu đến đối tượng hiện tại của **class**

Sử dụng từ khóa **this** có thể tham chiếu đến các thành viên của lớp như constructors, variables và methods

Từ khóa **this** được sử dụng bên trong một phương thức hoặc constructor

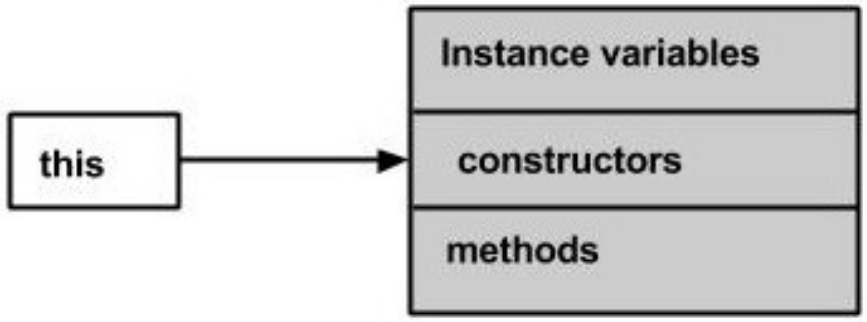


Từ khóa this

this là từ khóa được sử dụng để tham chiếu đến đối tượng hiện tại của **class**

Sử dụng từ khóa **this** có thể tham chiếu đến các thành viên của lớp như constructors, variables và methods

Từ khóa **this** được sử dụng bên trong một phương thức hoặc constructor




```
class Student {
    int age;
    Student() {
    this(20);
    }
    Student(int age){
        this.age = age;
    }
}
```

Từ khóa this

```
class ViDu {
        int test = 10;  // Biến Instance
    void printTest() {
    int test = 20; // Biến Local
    System.out.println("test = "+test); // In biến Local
    }
    public static void main(String args[]) {
    ViDu a = new ViDu();
    a.printTest();
    }
}
```

In biến Local, this chỉ đối tượng a

System.out.println("test = "+this.test);

METHOD (Phương thức hay hàm)

- **Method** (phương thức) là nhóm các câu lệnh để thực thi một thao tác.

- ***Hàm hay phương thức*** (method) trong Java là khối lệnh thực hiện các chức năng, các hành vi xử lý của lớp lên vùng dữ liệu.

```
[<Từ khóa truy cập>] <kiểu trả về> <tên phương thức>([<ds tham số>])
{
    // body of method
}
```

METHOD (Phương thức hay hàm)

- ***Các loại tham số trong phương thức***
  - ***Tham trị:*** Khi ta truyền một kiểu sơ cấp cho phương thức, nó sẽ truyền bằng giá trị. Vì vậy những gì xảy ra với tham số trong phương thức, khi ra khỏi phương thức sẽ hết tác dụng.

METHOD (Phương thức hay hàm)

```
class ViDu {
    void tinhToan(int i, int j) {
        i *= 2;
        j /= 2;
    }
}
class UngDung {
    public static void main(String  [ ] args) {
    ViDu o = new ViDu();
    int a = 15, b = 20;
    System.out.println("a và b trước khi gọi : "+a+ " "+b);
    o.tinhToan(a, b);
    System.out.println("a và b sau khi gọi : "+a+" "+b);
    }
}
```

METHOD (Phương thức hay hàm)

- ***Các loại tham số trong phương thức***
  - ***Tham chiếu:*** Khi ta truyền một đối tượng (kiểu phức hợp) cho phương thức, nó sẽ truyền bằng tham chiếu. Vì vậy, thay đổi ở đối tượng bên trong phương thức ảnh hưởng đến đối tượng dùng làm tham chiếu.

METHOD (Phương thức hay hàm)

```
class ViDu {
    int a, b;
    ViDu (int i, int j) {      a = i;    b = j;    }
    void tinhToan(ViDu o) {   o.a *= 2;    o.b /= 2;     }
}
class UngDung {
    public static void main(String args[]) {
    ViDu o = new ViDu(15, 20);
       System.out.println("o.a và o.b trước khi gọi : "+o.a+" "+o.b);
    o.tinhToan(o);
    System.out.println("o.a và o.b sau khi gọi : "+o.a+" "+o.b);
    }
}
```

METHOD (Phương thức hay hàm)

- ***Kết quả trả về của phương thức:*** Phương thức có thể có kiểu trả về hoặc **void** (không trả về gì cả)

```
<<kiểu trả về>> <<tên phương thức>> ([danh sách tham số])
{
        // thân phương thức
}
```

METHOD (Phương thức hay hàm)

```
public class Employee{
    public String fullname;
    public double salary;

    public void input(){…}
    public void output(){…}

    public void setInfo(String fullname, double salary) {
        this.fullname = fullname;
        this.salary = salary;
    }

    public double incomeTax(){
        if(this.salary < 5000000){
            return 0;
        }
        double tax = (this.salary - 5000000) * 10/100;
        return tax;
    }
}
```

Kiểu trả về là Void nên thân phương thức không có lệnh Return giá trị

Kiểu trả về là Double nên thân phương thức có lệnh Return số thực

HÀM THIẾT LẬP (CONSTRUCTOR)

- Là hàm thành phần của class, dùng để thiết lập các giá trị ban đầu cho các thuộc tính, hàm thiết lập có các đặc điểm sau:

  + Trùng tên với class
  + Thuộc tính truy xuất: public
  + Không có kiểu trả về
  + Tự động chạy khi khởi tạo đối tượng thuộc lớp

- Hàm thiết lập sao chép: là 1 hàm thiết lập dùng để sao chép đối tượng

HÀM THIẾT LẬP (CONSTRUCTOR)

```
class DIEM {
    int x; int y;
    public DIEM() {
            x = 0; y = 0;
    }
    public DIEM(int x, int yy) {
            this.x = x;
            y = yy;
    }
    public DIEM(DIEM mm) {
            this.x = mm.x;
            this.y = mm.y;
    }
    public void xuat() {
            System.out.println("Toa do: "+x+" "+y);
    }
}
```

Hàm thiết lập

Từ khóa **this** tham chiếu đến đối tượng hiện hành. Phân biệt thuộc tính x của đối tượng với tham số x.

Hàm thiết lập sao chép

HÀM THIẾT LẬP (CONSTRUCTOR)

```
class DIEM {
    int x; int y;
    public DIEM() {
            x = 0; y = 0;
    }
    public DIEM(int x, int yy) {
            this.x = x;
            y = yy;
    }
    public DIEM(DIEM mm) {
            this.x = mm.x;
            this.y = mm.y;
    }
    public void xuat() {
            System.out.println("Toa do: "+x+" "+y);
    }
}
```

```
public static void main(String args[])
{
    DIEM d1=new DIEM();
    d1.xuat();
    DIEM d2=new DIEM(7,9);
    d2.xuat();
    DIEM d3=new DIEM(d2);
    d3.xuat();
}
```

PHƯƠNG THỨC GET/SET

- Dùng để truy cập đến các biến của đối tượng
  + Phương thức get: lấy dữ liệu của thuộc tính
  + Phương thức set: gán dữ liệu cho thuộc tính
- Ví dụ tạo phương thức get/set cho thuộc tính **name**:

```
public void setName(String newname)
{
        name = newname;
}
public String getName()
{
        return name;
}
```

VÍ DỤ

```
class HinhChuNhat
{
    private float dai;
    private float rong;
    public void setDai(float newdai)
    {
        dai = newdai;
    }
    public float getDai()
    {
        return dai;
     }
    public void setRong(float newrong)
    {
        rong = newrong;
    }
    public float getRong()
    {
        return rong;
     }
}
```

```
class Program
{
    public static void main(String args[])
    {
        HinhChuNhat hcn1 = new HinhChuNhat();
        hcn1.setDai(5);
        hcn1.setRong(
      3);
        hcn1.xuat();
    }
}
```

THUỘC TÍNH STATIC

- Khi các đối tượng của lớp cần truy xuất chung 1 thuộc tính của chúng, thì hãy thiết kế thuộc tính static.

- Ví dụ: Lớp SinhVien có cùng số lượng sinh viên, thì thuộc tính số lượng sinh viên là thuộc tính static.

- Thuộc tính static thường được gán giá trị mặc định ban đầu

PHƯƠNG THỨC STATIC

- Khi đối tượng gọi phương thức của nó thì theo quy tắc:

  <tên đối tượng>.<tên phương thức(thamsố)>;

- Có một số phương thức thực hiện không cần có đối tượng thì khai báo chúng là phương thức static.

- Quy tắc gọi phương thức static là:

  <tên lớp>.<tên phương thức(tham số)>;

- Phương thức static chỉ truy xuất được các thuộc tính/phương thức static.

VÍ DỤ

```
public class Cat
{
   private static int count = 0;
   public Cat()
    {
       count++;
    }
    public static void HowManyCats()
    {
       System.out.println(count+" cats");
    }
}
```

```
public static void main(String[] args)
{
   Cat.HowManyCats();
   Cat mun = new Cat();
   Cat.HowManyCats();
   Cat muop = new Cat();
   Cat miu = new Cat();
   Cat.HowManyCats();
   }
```

BÀI TẬP

1. Thiết kế lớp DIEM bao gồm các thuộc tính tọa độ x,y, các phương thức get, set, nhập/ xuất/ di chuyển điểm. Di chuyển điểm có 2 tham số x' và y', x = x + x' và y = y + y'

2. Thiết kế lớp HINHCHUNHAT bao gồm các thuộc tính chiều dài, chiều rộng, các phương thức get,set, nhập/xuất/tính diện tích/tính chu vi hình chữ nhật

3. Thiết kế lớp HINHTRON bao gồm thuộc tính bán kính, các phương thức get,set, nhập/xuất/tính diện tích/tính chu vi hình tròn

4. Thiết kế lớp SINHVIEN bao gồm các thuộc tính: mã sinh viên, họ tên, lớp, điểm môn 1, điểm môn 2, điểm môn 3, các phương thức get, set, nhập, xuất, tính điểm trung bình, xếp loại

*** Các lớp đều có hàm thiết lập (có tham số và không tham số)